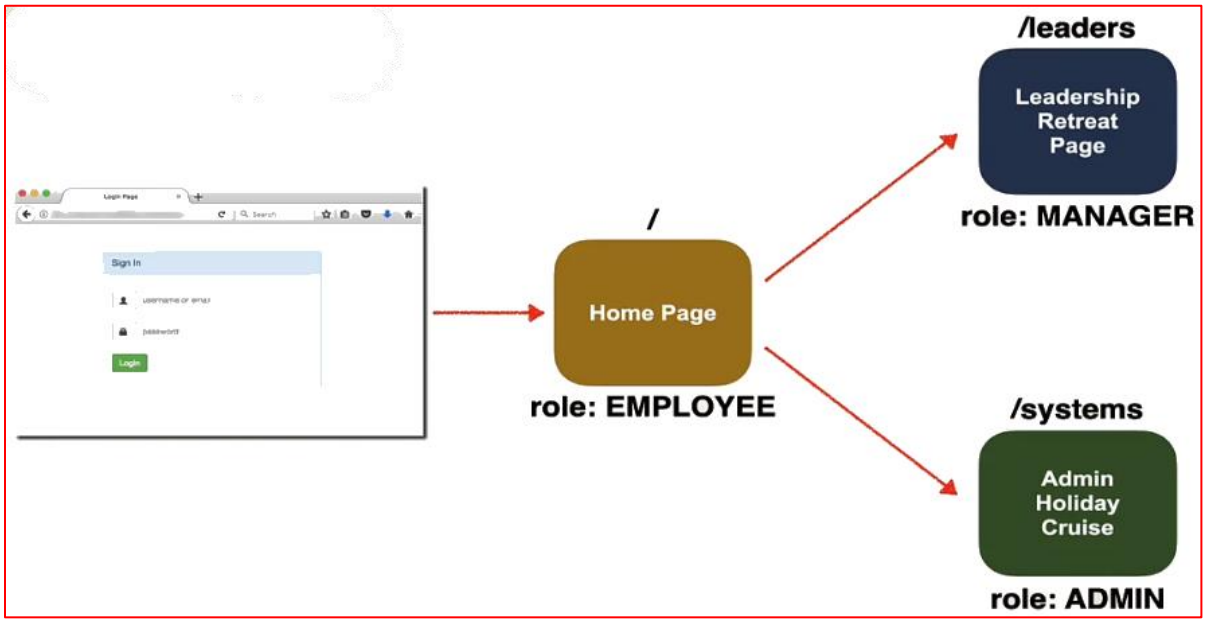
Spring MVC Security - Restrict URLs Based on Roles – Overview

In this lesson, we will explore how to control access to specific areas of a web application by defining user roles. Utilizing the example of a typical web portal, we will see how different user roles such as EMPLOYEE, MANAGER, and ADMIN can grant or restrict access to various parts of the application.

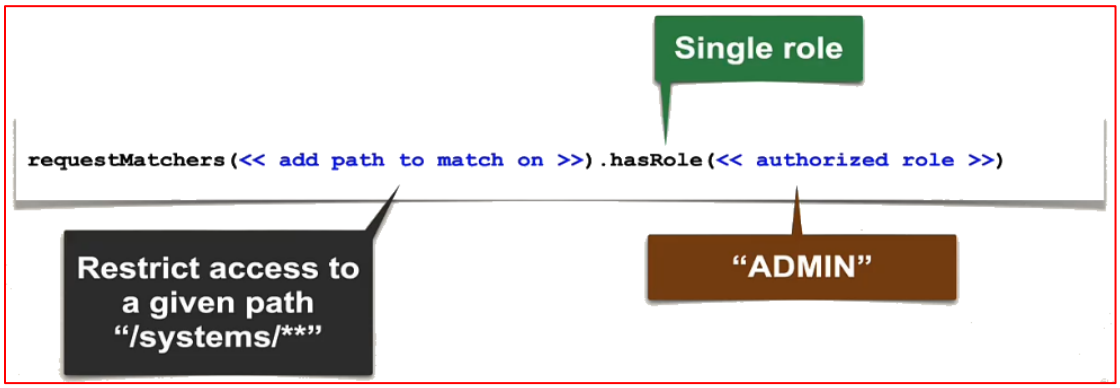
Let's begin by understanding our structure:

- **Login Page:** This will lead the user to the Home Page after successful authentication.
- **Home Page:** Accessible only to users with the EMPLOYEE role.
- **Leadership Retreat Page (`/leaders`):** Exclusive to users having the MANAGER role.
- **Admin Holiday Cruise Page (`/systems`):** A special section reserved for ADMIN users.



Development Process:

1. **Create Controller Code and View Pages:** We must first design the necessary controllers and views to represent the different sections. This will include pages for the Home Page, Leadership Retreat, and Admin Holiday Cruise. This process is standard Spring MVC work, with no security-specific coding required at this stage.
2. **Restrict Access Based on Roles:** Next, we will enforce role-based access control. This will be done through the Spring Security Java configuration.
    - **General Syntax:** Using request matchers, we will specify the path to match on, followed by the `.hasRole` method, and provide the authorized role. Example: restrict access to the path `/systems/**` for the ADMIN role.



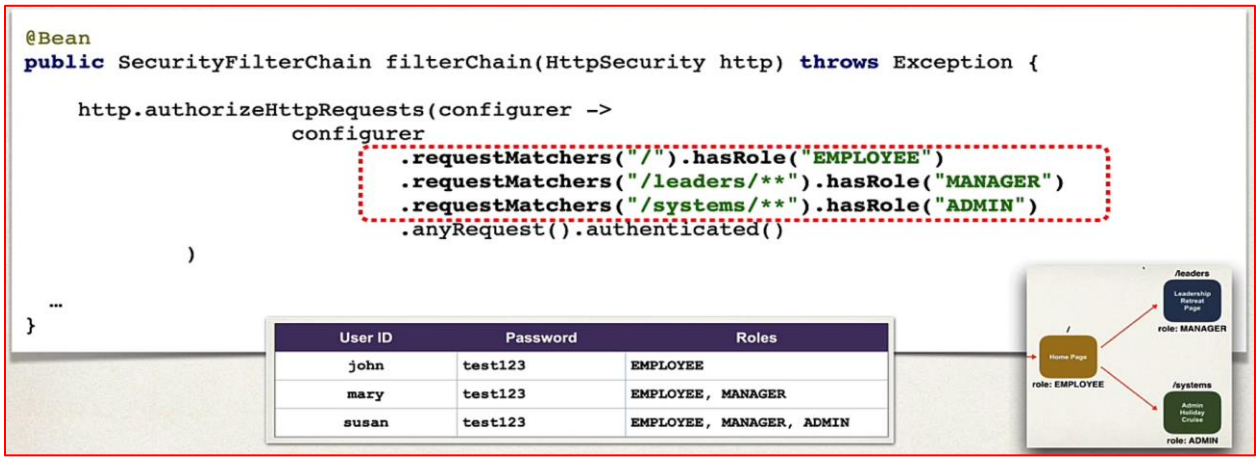
    - **Handling Multiple Roles:** For multiple roles, we can use `.hasAnyRole` and provide a comma-delimited list of roles. This will authorize access for any role in the given list.



Practical Examples:

- **EMPLOYEE Role:** To restrict the root path to the EMPLOYEE role, use `requestMatchers("/").hasRole("EMPLOYEE").`
- **MANAGER Role:** To secure the Leadership Retreat Page, apply `requestMatchers("/leaders/**").hasRole("MANAGER").`
- **ADMIN Role:** For the special Admin Holiday Cruise, employ `requestMatchers("/systems/**").hasRole("ADMIN").`

In summary, so far, we've seen how we can define access rules for different parts of our application based on specific user roles. By adhering to these principles, we can effectively control user access, enhancing the security and functionality of our application.



In the upcoming coding lesson, we'll dive into the actual code to see these concepts in action.